

Capítulo 2: Aprendizaje Supervisado

Universidad de Salamanca

Facultad de Ciencias

Programación Avanzada

Diego M. Jiménez-Bravo · dmjimenez@usal.es

- 1 Introducción
- 2 2.1 Regresión
- 3 2.2 Clasificación

1 Introducción

2 2.1 Regresión

3 2.2 Clasificación

¿Qué es el aprendizaje supervisado?

En el aprendizaje supervisado disponemos de:

- **Datos:** características de entrada, que denotamos X .
- **Etiquetas:** las respuestas correctas asociadas a cada dato, que denotamos y .
- Un **objetivo**: aprender la relación $X \rightarrow y$ a partir de ejemplos ya resueltos.

Es como aprender con un “supervisor” que te dice, durante el entrenamiento, si tu respuesta a cada ejemplo es correcta o no.

Existen dos grandes tipos de problemas, según la naturaleza de la etiqueta y que queremos predecir:

Regresión (valor continuo)

- Precio de una vivienda
- Temperatura de mañana

Clasificación (categoría)

- ¿Es un correo *spam*?
- ¿La imagen es un gato, un perro o un pájaro?

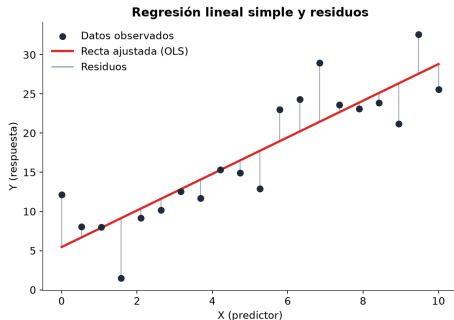
Contenidos

- 1 Introducción
- 2 2.1 Regresión**
- 3 2.2 Clasificación

Regresión lineal

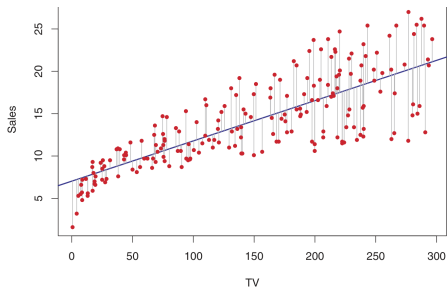
La regresión es el problema de predecir un **valor continuo** Y a partir de una o más variables predictoras X . La regresión lineal asume que existe una relación aproximadamente lineal entre ambas.

- **Simple**: un único predictor, $Y \approx \beta_0 + \beta_1 X$.
- **Múltiple**: un coeficiente por cada característica disponible, $Y = \beta_0 + \beta_1 X_1 + \dots + \beta_p X_p + \epsilon$.
- β_0 (**intercepto**) y $\beta_1, \dots, \beta_p$ (**pesos**) son lo que el modelo aprende de los datos. En un espacio de p dimensiones, el modelo representa geoméricamente un **hiperplano**.



Ejemplo real: dataset *Advertising*

Un ejemplo clásico de la literatura es el conjunto de datos *Advertising*, donde se ajusta una recta que predice las ventas (Sales) a partir de la inversión en publicidad en TV (TV). Cada punto gris vertical representa un **residuo**: la distancia entre el valor observado y el que predice la recta.



Fuente: James, Witten, Hastie & Tibshirani (2013), *An Introduction to Statistical Learning*, Fig. 3.1. Libro de libre distribución (statlearning.com).

Mínimos cuadrados ordinarios (OLS)

El método de **mínimos cuadrados ordinarios** (OLS) es el enfoque estándar para entrenar modelos lineales.

- Un **residual** $e_i = y_i - \hat{y}_i$ es la diferencia entre el valor real y el predicho.
- OLS busca los parámetros que hacen que esos residuales, elevados al cuadrado y sumados, sean lo más pequeños posible (a esta suma se le llama RSS).
- A diferencia de otros modelos, la regresión lineal tiene una **solución exacta**: existe una fórmula que calcula directamente los mejores parámetros en un único paso, sin necesidad de ensayo y error.

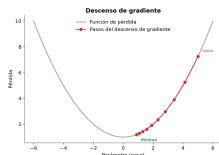
Ejemplo: con solo unas pocas casas (m^2 y precio), OLS encuentra en un único cálculo la recta que mejor resume esa relación.

Descenso de gradiente

Para conjuntos de datos masivos o modelos con miles de parámetros, resolver la ecuación exacta de OLS es demasiado costoso. El **descenso de gradiente** es el algoritmo de optimización iterativo que impulsa gran parte del aprendizaje automático moderno.

- El algoritmo parte de pesos aleatorios y los va ajustando poco a poco, dando pequeños pasos en la dirección que más reduce el error.
- La **tasa de aprendizaje** determina el tamaño de cada paso: si es muy alta, el algoritmo puede “pasarse” y no llegar a converger; si es muy baja, el entrenamiento será lentísimo.
- Variantes: *Batch* (usa todos los datos en cada paso), *Stochastic* (SGD, una sola muestra por paso) y *Mini-batch* (un grupo pequeño, el más usado en la práctica).

Ejemplo: es como bajar una montaña con niebla: en cada paso sientes la pendiente bajo tus pies y avanzas hacia donde más desciende, hasta llegar al valle.



Cuando los datos presentan curvas, se pueden extender los modelos lineales mediante **ingeniería de características**: se añaden potencias de los predictores originales (X^2 , X^3 , ...) como nuevas columnas de entrada.

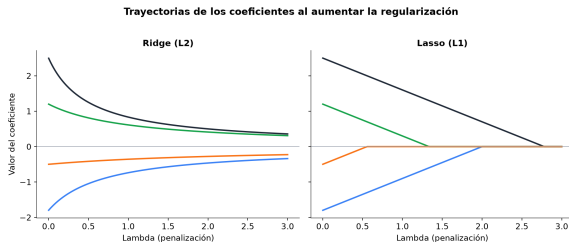
Aunque el modelo resultante es no lineal respecto a las variables originales, sigue siendo **lineal en sus parámetros**, así que se puede seguir entrenando con OLS o descenso de gradiente sin cambios.

El riesgo: aumentar el grado del polinomio aumenta la flexibilidad del modelo, lo que puede llevarlo a memorizar el ruido de los datos de entrenamiento (*overfitting*) en vez de captar la tendencia real.

Regularización: *Ridge* y *Lasso*

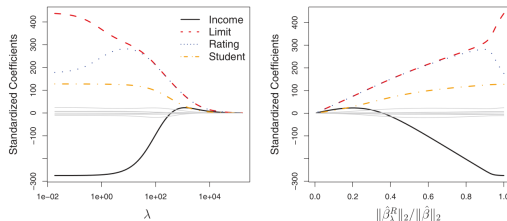
Las técnicas de regularización añaden una penalización a la función de coste por tener pesos grandes, forzando al modelo a ser más simple y a generalizar mejor.

- **Ridge**: encoge todos los coeficientes hacia cero, sin llegar nunca a eliminarlos; funciona muy bien cuando hay muchos predictores relacionados entre sí.
- **Lasso**: penaliza de forma algo distinta y puede llevar algunos coeficientes exactamente a cero, haciendo así una **selección automática de variables**.
- **Elastic Net**: combina ambas penalizaciones.



Ridge en datos reales: dataset *Credit*

Al aumentar el valor de λ (la fuerza de la penalización), los coeficientes se encogen progresivamente hacia cero. Esto reduce la **varianza** del modelo a cambio de un pequeño aumento del **sesgo** — exactamente el equilibrio que buscamos al regularizar.



Fuente: James, Witten, Hastie & Tibshirani (2013), An Introduction to Statistical Learning, Fig. 6.4.

- **MSE/RMSE:** penalizan fuertemente los errores grandes; el RMSE se expresa en las mismas unidades que Y , lo que facilita interpretarlo.
- **MAE** (error absoluto medio): el error medio “típico”, más robusto ante valores atípicos que el MSE.
- R^2 : qué proporción de la variación de Y logra explicar el modelo. 1 = ajuste perfecto; 0 = tan bueno como predecir siempre la media.
- **Análisis de residuos:** si al graficar los errores del modelo aparece un patrón claro en vez de ruido aleatorio, es señal de que el modelo lineal se está quedando corto.

- 1 Introducción
- 2 2.1 Regresión
- 3 2.2 Clasificación**

Clasificación: predecir categorías

En clasificación, el objetivo es predecir una **categoría** (clase) en lugar de un valor continuo. Algunos ejemplos típicos: clasificación de correos (*spam* vs. no *spam*), clasificación médica (enfermo vs. sano) o clasificación de especies (gato vs. perro).

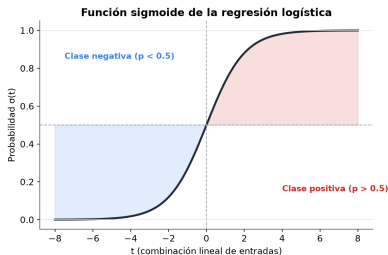
A diferencia de la regresión lineal, la clasificación comienza estimando la **probabilidad** de que una muestra pertenezca a una categoría concreta.

Regresión logística: la función sigmoide

La regresión logística utiliza la **función sigmoide** para convertir una combinación lineal de las características de entrada en una **probabilidad**, un valor siempre entre 0 y 1:

$$\sigma(t) = \frac{1}{1 + e^{-t}}$$

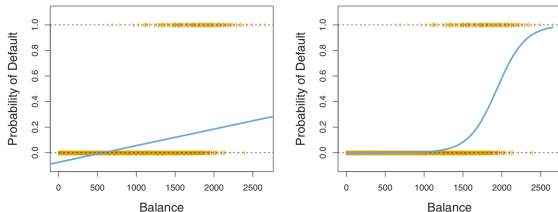
Cuanto mayor es t , más se acerca la salida a 1 (clase positiva); cuanto menor, más se acerca a 0. El modelo se entrena ajustando sus pesos, mediante descenso de gradiente, para que las probabilidades que predice se parezcan lo máximo posible a las etiquetas reales (0 o 1), penalizando con fuerza las predicciones que son seguras pero incorrectas.



¿Por qué no usar regresión lineal?

La regresión lineal puede predecir probabilidades menores que 0 o mayores que 1, lo cual no tiene sentido. La regresión logística, gracias a la sigmoide, mantiene la salida siempre dentro del rango $[0, 1]$.

El siguiente ejemplo compara ambos ajustes sobre el conjunto de datos *Default* (impago de crédito): a la izquierda, la regresión lineal produce probabilidades absurdas; a la derecha, la regresión logística siempre produce probabilidades válidas.

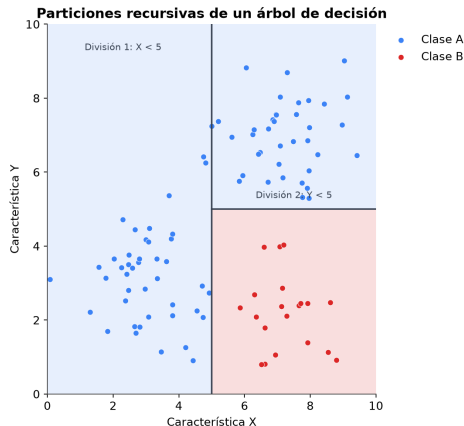


Fuente: James, Witten, Hastie & Tibshirani (2013), An Introduction to Statistical Learning, Fig. 4.2.

Árboles de decisión

Los árboles particionan el espacio de entrada de forma recursiva en “cajas” rectangulares, siguiendo un enfoque de “divide y vencerás”.

- **Construcción:** el árbol hace preguntas binarias sucesivas (“¿esta variable supera este valor?”), eligiendo en cada paso la que mejor separa las clases.
- **Poda** (*pruning*): para evitar el sobreajuste, se eliminan ramas innecesarias basándose en pruebas estadísticas o en un conjunto de validación.

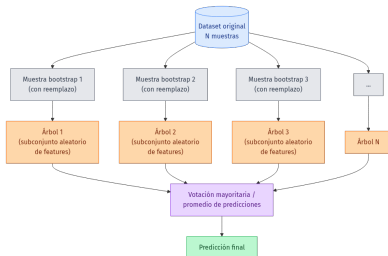


Ejemplo: “¿el email contiene la palabra ‘gratis’? → sí → ¿tiene más de 3 enlaces? → sí → *spam*”.

Bosques aleatorios (*Random Forests*)

Un bosque aleatorio es un conjunto de muchos árboles de decisión, diseñado para reducir la varianza y mejorar la robustez frente a un único árbol.

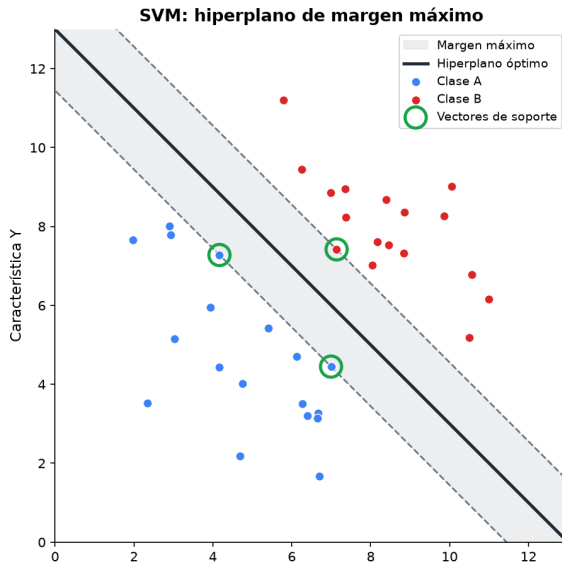
- **Bagging**: cada árbol se entrena con una muestra *bootstrap* distinta (obtenida por muestreo con reemplazo del conjunto original).
- **Decorrelación**: en cada división solo se considera un subconjunto aleatorio de *features*, para asegurar que los árboles sean diversos entre sí.
- Las predicciones individuales se combinan mediante **votación mayoritaria** (clasificación) o **promedio** (regresión).



Máquinas de vectores de soporte (SVM)

Este modelo busca el **hiperplano** que separa las clases con el **margen máximo** posible.

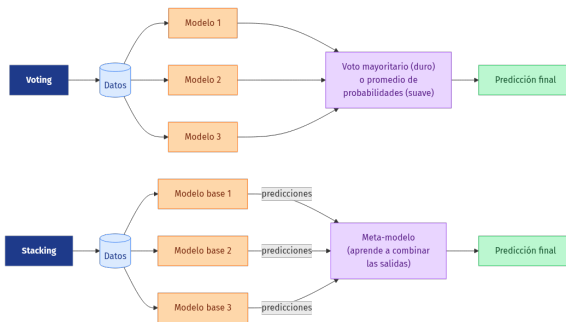
- Los **vectores de soporte** son las únicas muestras que determinan la posición de la frontera; mover cualquier otro punto no altera el modelo.
- Con datos no separables linealmente, el **margen blando** permite algunos errores, y el **truco del kernel** mapea implícitamente los datos a un espacio de más dimensiones donde sí son separables.



Voting y stacking

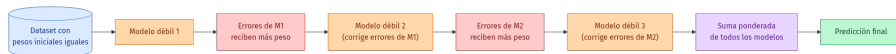
Los métodos de ensamble combinan varios modelos para obtener una predicción superior a la de cualquiera de ellos por separado.

- **Voting**: combina modelos independientes mediante una regla fija — voto mayoritario (“duro”) o promedio de probabilidades (“suave”).
- **Stacking**: entrena un **meta-modelo** que aprende, a partir de datos, cómo combinar las salidas de los modelos base — una regla aprendida, no fija.



A diferencia del entrenamiento en paralelo de los bosques aleatorios, el *boosting* entrena modelos de forma **secuencial**: cada nuevo modelo se centra en corregir los errores cometidos por el anterior, dando más peso a las instancias mal clasificadas. La predicción final combina todos los modelos mediante una suma ponderada.

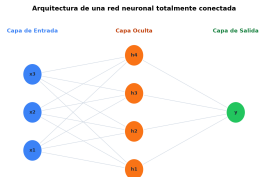
Ejemplos modernos y muy usados en competiciones de datos: **XGBoost** y **Gradient Boosting**.



Redes neuronales

Las redes neuronales representan el aprendizaje de representaciones sucesivas a través de capas de filtrado.

- **Arquitectura:** una capa de entrada, una o varias capas ocultas (densamente conectadas) y una capa de salida.
- **Funciones de activación:** introducen no linealidad para poder aprender patrones complejos. **ReLU** es el estándar en capas ocultas; **Softmax** se usa en la capa de salida para clasificación de múltiples clases.
- **Retropropagación** (*backpropagation*): el mecanismo central de aprendizaje. Propaga el error desde la salida hacia atrás, capa por capa, ajustando los pesos de la red para reducir la pérdida total.



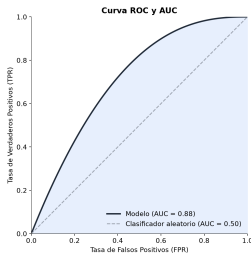
Igual que en regresión, es obligatorio dividir los datos en particiones independientes para medir bien la generalización:

- **Entrenamiento, validación y prueba:** un conjunto para ajustar pesos, otro para elegir hiperparámetros y un tercero, intacto, para la evaluación final.
- **Validación cruzada de K pliegues:** la estrategia recomendada cuando los datos son escasos.
- **Estratificación:** en clasificación, cada partición debe mantener la proporción original de clases, sobre todo si están desbalanceadas.

Evaluación: métricas de éxito

Las mismas métricas vistas en el capítulo anterior (exactitud, precisión, *recall*, F1, ROC-AUC) se aplican aquí, y son igual de importantes:

- **Exactitud** puede ser engañosa en datos desbalanceados: si el 90% de las muestras son de una clase, predecir siempre esa clase da 90% de exactitud sin haber aprendido nada.
- Por eso conviene mirar también **precisión** y **recall**, sobre todo cuando el coste de un error depende de qué clase se confunde.



La evaluación no ocurre solo al final: debe guiar todo el proceso de desarrollo.

- **Superar una línea de base:** antes de complicarse con modelos avanzados, hay que comprobar que se supera algo muy simple (por ejemplo, predecir siempre la clase más común).
- **Curvas de aprendizaje:** graficar el error de entrenamiento y validación a lo largo del entrenamiento revela el punto exacto en que aparece el sobreajuste.
- **Parada temprana** (*early stopping*): interrumpe el entrenamiento automáticamente cuando el modelo deja de mejorar en validación.
- **Análisis de errores:** revisar a mano los casos donde falla el modelo ayuda a entender qué lo está confundiendo.

- En **regresión** predecimos un valor continuo; en **clasificación**, una categoría.
- **OLS** da una solución exacta; el **descenso de gradiente** escala a datos masivos.
- La **regularización** (*Ridge/Lasso*) evita el sobreajuste penalizando los pesos grandes.
- La **regresión logística** extiende la idea de la regresión lineal a probabilidades mediante la sigmoide.
- **Árboles**, **Random Forest** y **SVM** son alternativas clásicas de clasificación, cada una con sus ventajas.
- Los métodos de **ensamble** (*voting, stacking, boosting*) combinan modelos para mejorar el resultado.
- Las **redes neuronales** aprenden representaciones en capas mediante retropropagación.
- Evaluar bien — con las particiones y métricas adecuadas — es tan importante como entrenar bien.

Este material se distribuye bajo licencia

Creative Commons Reconocimiento-NoComercial 4.0

(CC BY-NC 4.0)

Eres libre de:

- **Compartir:** copiar y redistribuir el material en cualquier medio o formato.
- **Adaptar:** remezclar, transformar y crear a partir del material.

Por ejemplo: usarlas en clase, subirlas al aula virtual, traducirlas o reutilizar una figura en tus apuntes.

Con estas condiciones:

- **Reconocimiento (BY):** debes citar la autoría, enlazar a la licencia e indicar si has hecho cambios.
- **NoComercial (NC):** no puedes usar el material con fines comerciales.

No puedes añadir restricciones que impidan a otros hacer lo que la licencia permite.

creativecommons.org/licenses/by-nc/4.0/deed.es

Capítulo 2: Aprendizaje Supervisado

Universidad de Salamanca

Facultad de Ciencias

Programación Avanzada

Diego M. Jiménez-Bravo · dmjimenez@usal.es